

TileMEM / TilePO V0.1 Technical Report

Subtitle: BF16 profile-guided tile-level placement/admission for MoE serving **Date:** 2026-06-12 **Project:** TileMEM
System: TilePO, Tile-level Placement Optimization **Release:** v0.1-priority-2026-06-11

Abstract

TileMEM is an open MoE serving optimization project. TilePO is its V0.1 algorithm/system: a BF16 profile-guided tile-level placement/admission path for high-throughput mixture-of-experts (MoE) serving. The core idea is to stop treating an MoE expert as the smallest memory-management unit. Instead, TilePO describes expert weights as memory tiles, uses profile evidence to decide which tiles should be admitted or retained in VRAM, and keeps the rest recoverable through a DRAM-backed fallback path. This makes the VRAM/DRAM hierarchy an explicit optimization target while preserving a fair KT-native BF16 execution comparison.

V0.1 evaluates TilePO against KT expert-level placement under the same expert budget. The released ablation matrix covers two workloads, five expert budgets, four placement policies, async planning on/off, three repeats, and five measured requests per row. The public manifest contains 210/210 successful real rows and passes the TilePO V0.1 gate. In this matrix, the best TilePO policy improves throughput by 10.59%-31.42% over KT and reduces p95 latency by 6.93%-29.14%, depending on workload and expert budget. The result supports TilePO as a practical placement/admission system for the tested BF16 MoE serving setting, not as a universal claim across all models, GPUs, workloads, or serving stacks.

1. Motivation: Why Tile Memory Instead Of Expert-Only Placement

MoE inference differs from dense-model inference because only a subset of experts is active for each token, and the active subset changes with prompt shape, output length, routing distribution, and memory budget. A coarse serving system usually asks a binary question: is this expert resident on GPU or not? That policy is simple, but it makes the expert the smallest unit of memory planning. When VRAM is limited, expert-only placement forces the system to keep or evict large chunks even when profile evidence says only part of the expert is hot for the current serving regime.

TilePO uses a finer memory unit. A tile is a stable slice of an expert weight projection, identified by layer, expert, projection group, shard id, and tensor range. This unit is still large enough to map cleanly onto serving kernels and runtime metadata, but small enough to express hot/cold differences inside the expert set. The intended effect is straightforward: keep hot tiles or hot experts close to the GPU fast path, let cold tiles occupy cheaper DRAM-backed residency, and use profile evidence to avoid paying tile overhead where coarse placement is already better.

The important systems point is that TilePO is not "always split everything into the smallest tile." Tile granularity is a planning lever. V0.1 exposes coarse, fine, and hybrid policies precisely because fine-grained memory control can help some regimes while hurting others through metadata, launch, grouping, or planning overhead. The V0.1 claim is therefore about profile-guided policy selection over the VRAM/DRAM hierarchy, not about fine tiles being universally dominant.

2. Core Implementation Architecture

TilePO is implemented as a compact serving artifact around a typed planning pipeline.

1. **. tmem plan interface.** A plan declares the model shape, workload label, tile policy, memory budget, precision policy, runtime mode, and backend fallback order. This keeps model-specific configuration outside the core scheduler.

2. **Typed MIR.** The compiler lowers . tmem into typed objects such as TileId, TileIR, RouteIR, ResidencyIR, PrecisionIR, and ScheduleIR. This makes tile identity, byte size, residency budget, precision, and fallback behavior explicit and checkable.

3. **Profile-guided placement/admission.** The admission layer reads real manifests, groups rows by workload and expert budget, compares TilePO against KT under the same expert budget, and emits a selected plan or a KT fallback plan. The production-facing rule is conservative: use TilePO only when the measured profile justifies it; otherwise return to KT.

4. **KT-native BF16 serving path.** V0.1 intentionally keeps the comparison on the BF16 KT-native path. TilePO changes placement/admission and planning, but it does not claim a completed full native CUDA replacement of KT/SGLang MoE kernels.

5. **Metrics and gates.** The artifact records throughput, p95, p99, GPU peak

memory, CPU/DRAM peak memory, readiness time, evidence level, repeat count, and success status. Verification scripts regenerate the ablation report from public manifests and fail if the gate is not satisfied.

This architecture is meant to be useful for real users: a user can provide a new MoE checkpoint, write or adapt a .tmem model plan, run the same-budget sweep, and let admission choose the best observed policy for that model and workload. TilePO does not assume one static algorithm can be optimal for every MoE model in the wild.

3. TilePO Policy Space In V0.1

V0.1 compares KT expert-level placement with three TilePO policy families and an async-planning switch.

- `kt_expert`: KT expert-level placement baseline.
- `tilepo_coarse`: coarse tile policy, intended to preserve grouped execution efficiency and throughput.
- `tilepo_fine`: fine tile policy, intended to expose more granular placement/admission choices.
- `tilepo_hybrid`: hot experts use coarse residency, while cold experts use finer admission.
- `async_planning=off/on`: whether planning is exposed synchronously or hidden behind the serving path.

These policies are not treated as marketing labels. They are ablation knobs used to answer a systems question: under the same expert budget, which placement granularity and planning exposure actually improves measured serving behavior?

4. Evaluation Setup

The V0.1 evaluation uses a BF16 KT-native serving path and compares against KT under identical expert budgets. The public matrix is intentionally small enough to be auditable and large enough to expose policy boundaries.

```
Workloads: mixed, long_context
Expert budgets: 2, 4, 6, 8, 10
Policies: kt_expert, tilepo_coarse, tilepo_fine, tilepo_hybrid
Async planning: off, on for TilePO policies
Repeats: 3
Request count: 5
Rows: 210 / 210 real success
Gate: PASS
Precision: BF16 / KT-native path
```

The metrics reported by the artifact include median throughput, p95 latency, p99 latency, GPU peak GiB, and CPU/DRAM peak GiB. TilePO reports memory metrics because VRAM/DRAM placement is part of the problem statement. V0.1 does not turn this into a universal memory-saving claim; memory behavior depends on the chosen policy, budget, and serving regime.

5. Results

The table reports the strongest TilePO result for throughput and the strongest TilePO result for p95 latency in each same-budget group. Throughput improvement is computed as $(\text{best TilePO tok/s} / \text{KT tok/s} - 1)$. p95 improvement is computed as $(1 - \text{best TilePO p95} / \text{KT p95})$.

Workload	Experts	Best tok/s policy	tok/s improvement	Best p95 policy	p95 improvement
long_context	2	hybrid, async off	+20.81%	coarse, async on	+15.81%
long_context	4	fine, async off	+26.42%	coarse, async on	+18.73%
long_context	6	coarse, async on	+28.31%	coarse, async on	+17.58%
long_context	8	coarse, async on	+31.16%	fine, async off	+29.14%
long_context	10	coarse, async off	+26.22%	fine, async on	+21.47%
mixed	2	hybrid, async off	+10.59%	hybrid, async off	+7.48%
mixed	4	hybrid, async on	+17.23%	hybrid, async on	+12.37%

Workload	Experts	Best tok/s policy	tok/s improvement	Best p95 policy	p95 improvement
mixed	6	coarse, async on	+21.95%	fine, async on	+15.98%
mixed	8	fine, async on	+31.42%	coarse, async on	+20.23%
mixed	10	fine, async off	+12.95%	fine, async on	+6.93%

The result is strongest for `long_context`, where all tested expert budgets show large throughput and p95 gains. The `mixed` workload is also positive across the tested budgets, but the winning policy changes more often. That is exactly the kind of behavior TilePO is designed to expose: the right policy is a measured property of a model/workload/budget configuration, not a fixed global rule.

6. Interpretation

The V0.1 evidence supports three conclusions.

First, same-budget placement policy matters. Even when KT and TilePO use the same expert budget, changing the placement/admission policy changes throughput and latency. This means the expert budget alone does not fully determine serving performance.

Second, tile-level planning is useful as a policy space, not as a blanket rule. Coarse policies often preserve throughput, fine policies help selected regimes, and hybrid policies can be competitive when hot and cold experts should be treated differently. The measured winner varies across workload and budget.

Third, admission should be conservative in production. TilePO should be used when profile evidence says it wins for the requested model and workload. If the measured profile does not justify TilePO, the scheduler should return to the KT path. This makes TilePO a deployable optimization layer rather than an always-on replacement.

7. Claim Boundaries

V0.1 makes a narrow claim and keeps several boundaries explicit.

- TilePO is a BF16 profile-guided tile-level placement/admission system for MoE serving.
- The public V0.1 matrix contains 210/210 successful real rows and passes its verification gate.
- TilePO outperforms KT expert-level placement in the tested same-budget `mixed` and `long_context` matrix.
- V0.1 does not claim universal wins across all models, GPUs, workloads, or serving systems.
- V0.1 does not claim full native CUDA replacement of KT/SGLang MoE kernels.
- V0.1 does not claim FP8 or MXFP4 serving-quality gains.
- V0.1 does not claim that fine-grained tile splitting alone explains every observed win.

This wording is intentional. TilePO is positioned as an open, verifiable systems artifact with strong evidence in its tested regime, not as an overbroad novelty claim that broad prior art could easily attack.

8. Reproducibility

Offline artifact verification does not require a GPU or model checkpoint:

```
git clone https://github.com/TerminusAkivili/TileMEM.git
cd TileMEM
python3 -m pip install -e .
bash examples/quickstart_offline.sh
```

Expected key output:

```
TilePO V0.1 ablation gate: PASS
Rows: 210/210
Groups: 70
```

Real BF16 serving evaluation requires a compatible MoE checkpoint and local KT/SGLang runtime environment:

```
export TILEMEM_MODEL_PATH=/path/to/moe/checkpoint
export TILEMEM_PLAN=configs/models/olmoe_1b_7b_example.tmem
export TILEMEM_WORKLOAD=mixed
export TILEMEM_EXPERTS=2,4,6,8,10
export TILEMEM_OUT_DIR=build/custom_model_run

bash examples/quickstart_custom_model.sh
```

9. Priority And Citation Record

TileMEM / TilePO V0.1 was publicly released as a priority artifact on 2026-06-11. The release does not depend on arXiv, ChinaXiv, OSF, or TechRxiv acceptance.

```
GitHub repository: https://github.com/TerminusAkvili/TileMEM
GitHub release: https://github.com/TerminusAkvili/TileMEM/releases/tag/v0.1-priority-2026-06-11
Zenodo DOI: https://doi.org/10.5281/zenodo.20646195
Zenodo concept DOI: 10.5281/zenodo.20646194
Software Heritage SWHID: swh:1:snp:073ee68e366c28f478e81db109056b68f9b146ab
Release tarball SHA256: 4592f09fb451c5d0fe998d9f4fb83ab774100ddba72dc580ef1c5772b7b70f3b
```

Recommended citation:

```
@software{tilemem_tilepo_v0_1_2026,
  title = {TileMEM TilePO v0.1: BF16 Profile-Guided Tile-Level Placement/Admission for MoE Serving},
  author = {TerminusAkvili},
  year = {2026},
  version = {v0.1-priority-2026-06-11},
  doi = {10.5281/zenodo.20646195},
  url = {https://github.com/TerminusAkvili/TileMEM}
}
```

10. Acknowledgements

TileMEM uses and studies ideas in the ecosystem around KTransformers, SGLang, and TileLang. The V0.1 artifact thanks those upstream projects for making MoE serving, serving-system integration, and tile-oriented kernel research easier to study in the open.